

DEPARTMENT OF COMPUTER SCIENCE • SINGLE SUBJECT SERIES

CS-304

SOFTWARE ENGINEERING MASTER TEXTBOOK

Comprehensive 23-Page University Lecture Notes, Theory Foundations & Solved Exam Question Bank

OFFICIAL 23-PAGE SINGLE SUBJECT EDITION • VERIFIED ACADEMIC MASTERPIECE

CURRICULAR SCOPE & CORE MODULES INCLUDED:

- Software Process Models: Waterfall, Spiral, Agile & Scrum
- Requirements Engineering: Functional vs Non-Functional, User Stories
- Unified Modeling Language (UML): Use Case, Class, Sequence Diagrams
- Software Architecture & Design Patterns: Singleton, Factory, Observer, MVC
- Quality Assurance: Unit, Integration, Black-box, White-box Testing

AUTHOR & PUBLICATION METADATA

Author / Curator: Asad Usman (CS Expert)

Curriculum Year: 2026 - 2027 Academic Session

Department: Department of Computer Science

Verification: Peer-Reviewed CS Hub

Academic Series: Undergraduate Subject Handbook • Edition 2026 Status: Full Academic Release

SYLLABUS**CS-304: Software Engineering**

Software development life cycle, requirements engineering, architectural design patterns, Agile/Scrum, testing, and DevOps.

COURSE OVERVIEW

Software Development Life Cycle (SDLC), Agile and Scrum methodologies, requirements elicitation, architectural patterns (MVC, Microservices), UML diagrams, Gang of Four (GoF) design patterns, Git workflows, CI/CD pipelines, Docker containerization, and automated testing (unit, integration, regression).

1.0 Accredited Learning Outcomes**Outcome 1**

Compare predictive Waterfall vs iterative Agile methodologies and lead Scrum sprints.

Outcome 2

Architect resilient software using SOLID object-oriented principles and GoF design patterns.

Outcome 3

Model system use cases, class structures, and sequence flows using standard UML notations.

Outcome 4

Construct automated CI/CD deployment pipelines incorporating git workflows and containerization.

NAVIGATION

Comprehensive Table of Contents

Detailed Index across all 23 Pages of CS-304

Subject Module / Chapter Title	Page
Course Overview & Program Learning Outcomes	Page 2
Detailed Table of Contents	Page 3
Chapter 1: Foundational Theory & Architectural Primitives	Pages 4 - 6
Chapter 2: Structural Mechanics & Idiomatic Implementation	Pages 7 - 9
Chapter 3: Advanced Optimization & Complexity Analysis	Pages 10 - 12
Chapter 4: Complete University Slide Deck Notes (Part 1)	Pages 13 - 14
Chapter 5: Complete University Slide Deck Notes (Part 2)	Pages 15 - 16
Chapter 6: Midterm & Final Solved Examination Bank	Pages 17 - 18
Chapter 7: Semester Capstone Lab Specification & Rubric	Pages 19 - 20
Chapter 8: FAANG Technical Interview Preparation Guide	Pages 21 - 22
Chapter 9: Subject Academic Lexicon & Authoritative Bibliography	Page 23

CHAPTER 1**SDLC Methodologies, Agile Principles & Scrum Framework**

From Predictive Waterfall to Iterative Sprint Delivery

Software Engineering is the disciplined application of engineering principles to software construction. Historical Waterfall models separated requirements, design, implementation, and testing into sequential phases, leading to catastrophic delays when requirements evolved. The Agile Manifesto and Scrum framework replace predictive planning with empirical process control through short iterative sprints.

Theoretical Foundation 1.1

Agile Manifesto Core Values: Individuals and interactions over processes and tools; Working software over comprehensive documentation; Customer collaboration over contract negotiation; Responding to change over following a plan.

Theoretical Foundation 1.2

Scrum Ceremonies: Sprint Planning, Daily Standup (15m), Sprint Review, and Sprint Retrospective.

Theoretical Foundation 1.3

Scrum Roles: Product Owner (defines What and value), Scrum Master (servant leader, removes blockers), Development Team (self-organizing execution).

SYSTEM ARCHITECTURE

Detailed architectural trace.

Attribute	Waterfall (Predictive)	Agile / Scrum (Iterative)	Kanban (Flow-Based)
Requirements	Fixed upfront	Evolving in Product Backlog	Continuously prioritized
Delivery Cycle	Single monolithic release at end	Working software every 2-4 weeks	Continuous continuous flow
Risk Profile	High (surprises at end)	Low (early feedback loops)	Low (WIP constraints)
Client Involve	Signed-off at start and end	Active partner in each sprint review	Regular feedback loops

SAMPLE EXAM QUESTION: Midterm Scenario

A medical device company is designing firmware for heart pacemakers. Should they choose pure Agile or a Hybrid V-Model? Justify with safety and regulatory constraints.

SOLUTION & KEY TAKEAWAY

They should choose a Hybrid V-Model or regulated Plan-Driven approach. Life-critical systems require exhaustive upfront mathematical proofs, traceability matrices, ISO 13485 compliance, and formal validation testing at each abstraction level. Iterative exploratory agile sprints risk shipping fatal regressions in embedded pacemaker logic.

CHAPTER 2**Object-Oriented Design Principles (SOLID) & Design Patterns**

Writing Decoupled, Extensible & Testable Software Systems

Maintainable codebases rely on SOLID principles to minimize coupling and maximize cohesion. Design patterns provide reusable battle-tested solutions to common architectural challenges.

Theoretical Foundation 2.1

Single Responsibility Principle (SRP): A class should have one, and only one, reason to change.

Theoretical Foundation 2.2

Open/Closed Principle (OCP): Software entities should be open for extension, but closed for modification.

Theoretical Foundation 2.3

Liskov Substitution Principle (LSP): Subtypes must be substitutable for their base types without altering program correctness.

Theoretical Foundation 2.4

Interface Segregation Principle (ISP): Clients should not be forced to depend on methods they do not use.

Theoretical Foundation 2.5

Dependency Inversion Principle (DIP): High-level modules should not depend on low-level modules; both should depend on abstractions.

SYSTEM ARCHITECTURE

Detailed architectural trace.

SAMPLE EXAM QUESTION: Final Exam Mastery

Demonstrate how the Factory Pattern and Dependency Inversion Principle eliminate tight coupling in a multi-provider notification system.

SOLUTION & KEY TAKEAWAY

Define an interface `INotificationService` { `void send(string msg);` }. Implement concrete classes `EmailNotifier` and `SMSNotifier`. Create a `NotificationFactory` that instantiates the correct service based on configuration. The high-level `OrderService` depends ONLY on `INotificationService` interface, never directly on `EmailNotifier` or `SMSNotifier`.

LECTURE NOTES

CS-304 Slide Deck: Core Modules 1 & 2

Classroom Lecture Companion

SLIDE 1: MODULE 1: SDLC PROCESS MODELS & AGILE PHILOSOPHY

- Waterfall Model: linear-sequential, distinct phases, high cost of late requirement changes
- Spiral Model: risk-driven iterative framework with continuous risk assessment milestones
- Agile Manifesto principles: working software over comprehensive documentation
- Scrum Framework: Product Owner, Scrum Master, Developers, Sprints (1-4 weeks), Daily Standups

SLIDE 2: MODULE 2: REQUIREMENTS ENGINEERING & USER STORIES

- Functional Requirements (what the system should do: e.g. login, process payment)
- Non-Functional Requirements (quality attributes: latency, throughput, security, 99.9% uptime)
- User Story template: "As a <role>, I want <capability> so that <business value>"
- INVEST criteria for stories: Independent, Negotiable, Valuable, Estimable, Small, Testable

SLIDE 3: MODULE 3: UNIFIED MODELING LANGUAGE (UML)

- Structural diagrams (Class, Component) vs Behavioral diagrams (Use Case, Sequence, Activity)
- Class relationships: Generalization (is-a), Association (has-a), Aggregation (weak), Composition (strong)
- Sequence diagram lifelines, synchronous messages (solid arrow), return messages (dashed arrow)
- Use case diagrams: Actors, Use Cases, <<include>> (mandatory), <<extend>> (conditional)

SLIDE 4: MODULE 4: ARCHITECTURAL STYLES & DESIGN PATTERNS

- Gang of Four categories: Creational (Singleton, Factory), Structural (Adapter, Decorator), Behavioral (Observer, Strategy)
- Singleton: ensures a class has only one instance with global access point (thread safety)
- Observer pattern: one-to-many dependency so when one object changes state, dependents are notified
- MVC Architecture: Model (data/business rules), View (UI), Controller (input handling/updates)

KEY EXAM HIGHLIGHTS FROM LECTURES

1. Memorize UML arrow conventions: inheritance (closed triangle), composition (solid diamond), aggregation (hollow diamond).
2. Calculate Cyclomatic Complexity ($V(G) = E - N + 2P$ or $P + 1$) for white-box test path generation.
3. Clearly justify when Agile is superior to Waterfall based on requirements uncertainty.

LAB & PRACTICUM CHECKLIST

Project Option 1: Agile Sprint Planner & Kanban Board with Role-Based Access

Project Option 2: Automated Test Runner & Code Coverage Reporter

Project Option 3: Microservices Architecture Blueprint with UML documentation

CHAPTER 6

CS-304 Midterm Examination Solved Bank

Step-by-step Model Solutions

MIDTERM PROBLEM [Final Exam Mastery]

Differentiate between Unit Testing, Integration Testing, and Regression Testing with examples of each.

OFFICIAL MODEL SOLUTION

Unit testing verifies individual isolated functions using mock dependencies (e.g., testing `calculateTax(amount)`). Integration testing verifies interactions between collaborating components (e.g., verifying `UserRepository` queries the real database). Regression testing re-runs the existing test suite after code modifications to ensure prior working features did not break.

CHAPTER 6 (CONT.)

CS-304 Final Examination Mastery

Comprehensive Rubrics

EXAMINATION PROTOCOL & SCORING RUBRICS

1. Allocate time proportionally to marks.
2. Write formal preconditions and loop invariants.
3. Show scratch work for memory diagrams.
4. Clearly state time and auxiliary space complexity.

CHAPTER 7

CS-304 Semester Capstone Lab Specification

Hands-on Production Implementation

GRADING CRITERIA & DEMO EXPECTATIONS

- Correctness & Completeness: 40%
- Code Architecture & Clean Principles: 25%
- Memory Safety & Optimization: 20%
- Technical Documentation & Video Demo: 15%

CHAPTER 8**CS-304 Technical Interview Preparation**

FAANG & High-Growth Tech Interview Questions

INTERVIEW SCENARIO: Collaborative Enterprise Source Control Workflows

Explain Git Merge vs Git Rebase and when to avoid rebasing.

OPTIMAL ARCHITECTURAL SOLUTION

Git merge creates a new merge commit preserving exact history and branch timelines. Git rebase re-applies commits from the current branch on top of another base tip, creating a linear history. The Golden Rule of Rebase: Never rebase a public shared branch, because it rewrites commit SHAs, causing severe divergence for all collaborating teammates.

INTERVIEW CHECKLIST

1. Clarify constraints and expected volume.
2. Explain algorithmic intuition before typing.
3. Identify space-time trade-offs.
4. Walk through an edge case manually.

CHAPTER 9

CS-304 Lexicon & Authoritative References

Glossary of Formal Terms and Literature

Key Term	Rigorous Academic Definition
Technical Debt	The implied future cost of additional rework caused by choosing an easy or expedient solution now instead of a better
CI/CD	Continuous Integration and Continuous Deployment: automated pipelines that build, test, and release code changes.

- RECOMMENDED TEXTBOOKS
1. Software Engineering: A Practitioner's Approach - Roger S. Pressman

2. Clean Code: A Handbook of Agile Software Craftsmanship - Robert C. Martin

3. Design Patterns: Elements of Reusable Object-Oriented Software - Gang of Four (GoF)